Complete RTL-to-GDSII Setup Guide Using OpenROAD (Ubuntu + WSL)

Overview

This document is a complete end-to-end guide for:

- Installing Ubuntu using WSL2 on Windows
- Installing OpenROAD-flow-scripts
- Installing required dependencies
- Setting up Yosys and OpenROAD
- Running RTL-to-GDSII flow
- Viewing layouts in KLayout
- Generating synthesis and physical design outputs
- Debugging common installation/runtime errors
- Organizing project outputs professionally

This guide is based on a real UART RTL-to-GDSII implementation using the SKY130HD PDK.

---

1. System Requirements

Recommended Hardware

Component	Recommended
RAM	16 GB or higher
CPU	Quad-core or higher
Storage	40+ GB free space
OS	Windows 10/11 with WSL2 or Native Ubuntu

---

2. Installing WSL2 and Ubuntu

Step 1: Open PowerShell as Administrator

Run:

```
wsl --install
```

Restart your system after installation.

Step 2: Install Ubuntu

Open Microsoft Store.

Install:

- Ubuntu 22.04 LTS

Launch Ubuntu.

Create:

- Username
- Password

Step 3: Verify WSL Installation

Run:

```
wsl -l -v
```

Expected:

```
Ubuntu    Running    Version 2
```

---

3. Updating Ubuntu

Run:

```
sudo apt update && sudo apt upgrade -y
```

---

4. Install Essential Packages

Run:

```
sudo apt install -y
build-essential
git
cmake
python3
python3-pip
curl
wget
clang
bison
flex
libreadline-dev
gawk
tcl-dev
libffi-dev
graphviz
xdot
pkg-config
python3-tk
libboost-system-dev
libboost-python-dev
libboost-filesystem-dev
zlib1g-dev
qtbase5-dev
libqt5svg5-dev
libgdal-dev
```

---

5. Install Docker

Docker is required for OpenROAD-flow-scripts.

Install Docker

```
sudo apt install docker.io -y
```

Enable Docker:

```
sudo systemctl enable docker
sudo systemctl start docker
```

Verify:

```
docker --version
```

---

6. Clone OpenROAD-flow-scripts

Create workspace:

```
mkdir ~/openroad_projects
cd ~/openroad_projects
```

Clone repository:

```
git clone --recursive https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git
```

Enter directory:

```
cd OpenROAD-flow-scripts
```

---

7. Build OpenROAD

Build Dependencies

Run:

```
sudo ./setup.sh
```

This step installs:

- OpenROAD
- Yosys

- Python dependencies
- Additional backend tools

This may take a long time.

---

8. Build OpenROAD Tools

Run:

```
./build_openroad.sh --local
```

Expected output:

```
Build complete
```

---

9. Verify OpenROAD Installation

Run:

```
./tools/install/OpenROAD/bin/openroad
```

Expected:

```
OpenROAD>
```

Exit:

```
exit
```

---

10. Install KLayout

KLayout is used to view:

- DEF

- GDSII
- Layout stages
- Routing
- Congestion
- Clock trees

Install:

```
sudo apt install klayout -y
```

Verify:

```
klayout
```

---

11. Install X11 GUI Support for WSL

If using Windows + WSL:

Install:

- VcXsrv OR XLaunch

Launch XLaunch before opening GUI tools.

Recommended settings:

- Multiple windows
- Start no client
- Disable access control

---

12. Set DISPLAY Variable

Run:

```
export DISPLAY=$(grep nameserver /etc/resolv.conf | awk '{print $2}'):0
```

To make permanent:

```
echo 'export DISPLAY=$(grep nameserver /etc/resolv.conf | awk '\''{print
$2}'\''):0' >> ~/.bashrc
source ~/.bashrc
```

---

13. Common GUI Error and Fix

Error

```
qt.qpa.xcb: could not connect to display
```

Cause

GUI server not running.

Solution

1. Start XLaunch/VcXsrv
2. Export DISPLAY variable
3. Retry OpenROAD or KLayout

---

14. Create RTL Project Structure

Recommended structure:

```
uart-rtl-to-gdsii/
├── rtl/
├── testbenches/
├── synthesis/
├── simulations/
├── openroad/
│   ├── synthesis/
│   ├── floorplan/
│   ├── placement/
│   ├── cts/
│   ├── routing/
│   ├── final/
│   ├── reports/
```

```
│   └── visualizations/
└── documentation/
```

---

15. Add RTL Files

Example:

```
rtl/
├── baud_gen.v
├── uart_tx.v
├── uart_rx.v
└── uart_top.v
```

---

16. Create OpenROAD Design Config

Go to:

```
cd OpenROAD-flow-scripts/flow/designs/sky130hd
```

Create project directory:

```
mkdir uart
```

Create:

```
config.mk
```

Example:

```
export DESIGN_NAME = uart_top

export PLATFORM = sky130hd

export VERILOG_FILES = $(DESIGN_HOME)/src/uart_top.v
$(DESIGN_HOME)/src/uart_tx.v
$(DESIGN_HOME)/src/uart_rx.v
```

```
$(DESIGN_HOME)/src/baud_gen.v

export SDC_FILE = $(DESIGN_HOME)/uart.sdc

export CLOCK_PORT = clk

export CLOCK_PERIOD = 20
```

---

17. Add RTL Files to src/

Copy files:

```
mkdir src
cp ~/uart_project/rtl/*.v src/
```

---

18. Create SDC File

Create:

```
uart.sdc
```

Example:

```
create_clock [get_ports clk] -period 20
set_input_delay 2 [all_inputs]
set_output_delay 2 [all_outputs]
```

---

19. Run RTL-to-GDSII Flow

Go to:

```
cd ~/openroad_projects/OpenROAD-flow-scripts/flow
```

Run:

```
make DESIGN_CONFIG=./designs/sky130hd/uart/config.mk
```

---

20. OpenROAD Flow Stages

Stage	Description
Synthesis	Converts RTL to gate-level netlist
Floorplanning	Defines chip dimensions
Placement	Places standard cells
CTS	Builds clock tree
Routing	Connects all nets
Fill	Adds fill cells
GDSII	Final fabrication layout

---

21. Important Output Files

File	Purpose
.v	Netlist
.odb	OpenDB database
.def	Physical design
.gds	Final layout
.sdc	Constraints
.spef	Extracted parasitics
.rpt	Reports

---

22. Viewing DEF Layouts

Example:

```
klayout ~/orfs_persistent/results/base/3_place.def
```

If DEF not present, use:

```
klayout ~/orfs_persistent/results/base/6_final.gds
```

---

23. Viewing Final GDSII

Run:

```
klayout ~/orfs_persistent/results/base/6_final.gds
```

This displays:

- Routed design
- Power rails
- Standard cells
- Metal layers
- Vias

---

24. Common KLayout Problem

Error

```
Macro not found in LEF file
```

Cause

DEF loaded without LEF libraries.

Solution

Usually safe to ignore for quick visualization.

For proper visualization:

- Load LEF technology files

- Or directly open GDSII instead

---

25. Common OpenROAD Errors

Error: nano not found

Cause

Nano editor missing.

Fix

```
sudo apt install nano -y
```

Alternative:

```
vim file.txt
```

Error: child killed: illegal instruction

Example

```
Error: cts.tcl, 86 child killed: illegal instruction
```

Cause

Usually:

- CPU virtualization issue
- Docker/WSL incompatibility
- Unsupported instruction set
- OpenROAD binary issue

Possible Fixes

1. Restart WSL:

```
wsl --shutdown
```

1. Restart Ubuntu
2. Rebuild OpenROAD
3. Use fewer threads
4. Skip problematic stage temporarily

Error: could not connect to display

Cause

No GUI server.

Fix

- Start XLaunch
- Export DISPLAY variable

Error: Unable to open GDS

Cause

Wrong file path.

Fix

Use:

```
ls
```

Verify file exists.

---

26. Generating Gate-Level Netlist Visualization

Open Yosys:

```
yosys
```

Read netlist:

```
read_verilog 1_2_yosys.v
```

Set top module:

```
hierarchy -top uart_top
```

Generate schematic:

```
show -format png -prefix uart_gatelevel uart_top
```

Generated:

```
uart_gatelevel.png
```

---

27. Generating Screenshots for GitHub

Recommended screenshots:

- Synthesized netlist
- Floorplan
- Placement
- Clock tree
- Routing
- Congestion map
- IR drop map
- Final routed chip

---

28. Important Reports to Save

Recommended reports:

```
2_floorplan_final.rpt
3_global_place.rpt
3_detailed_place.rpt
4_cts_final.rpt
5_route_drc.rpt
6_finish.rpt
```

---

29. Professional GitHub Organization

Recommended:

```
openroad/
├── synthesis/
├── floorplan/
├── placement/
├── cts/
├── routing/
├── final/
├── reports/
└── visualizations/
```

---

30. Suggested GitHub README Sections

Include:

1. Project overview
2. RTL architecture
3. Synthesis
4. Floorplanning
5. Placement
6. CTS
7. Routing
8. Final GDSII
9. Reports
10. Future work

---

31. Recommended Future Improvements

Possible future additions:

- DRC verification
- LVS verification
- Power optimization
- Multi-clock design
- Scan chain insertion
- STA optimization
- ASIC tapeout preparation
- Antenna fixing
- Timing closure optimization

---

32. Final Notes

By completing this flow, you successfully:

- Converted RTL into physical silicon layout
- Performed backend ASIC implementation
- Generated fabrication-ready GDSII
- Explored placement, CTS, routing, congestion, and IR-drop analysis
- Built a complete RTL-to-GDSII ASIC project using open-source tools

This workflow represents a real ASIC backend implementation pipeline similar to industrial physical design flows.